Semester:VII
Academic Year: 2026-2027
Class / Branch: BE IT-A
Subject:Recent Open Source Lab
Name of Instructor: Prof. Sachin Kasare.

Name: khushi anchalia
Student ID: 23104214

EXPERIMENT NO. 04

Aim: To examine various open-source quality assurance tools like Selenium, Katalon, Cucumber and identify steps for maintaining quality using Selenium.

Selenium is a powerful, open-source tool used for automating web browsers. It allows developers and testers to simulate user actions like clicking, typing, and navigating through web pages. Selenium is widely used for testing web applications, automating repetitive browser tasks, and web scraping.

Key Features:

1. Browser Automation: Automates actions in browsers like Chrome, Firefox, Safari, and more.
2. Cross-Browser Testing: Supports different browsers, enabling testing across various platforms.
3. Programming Languages: Can be used with multiple languages such as Python, Java, C#, Ruby, and JavaScript.
4. WebDriver: Selenium's main component, allowing interaction with web pages via real browsers.
5. Element Interaction: Allows locating elements using various strategies (e.g., by ID, name, XPath) and interacting with them (clicking, typing, etc.).
6. Open Source: Free to use with a large community for support.

Prerequisites:

- **Install Python**: Ensure Python is installed on their systems.
- **Install Selenium**: Run the following command in the terminal:

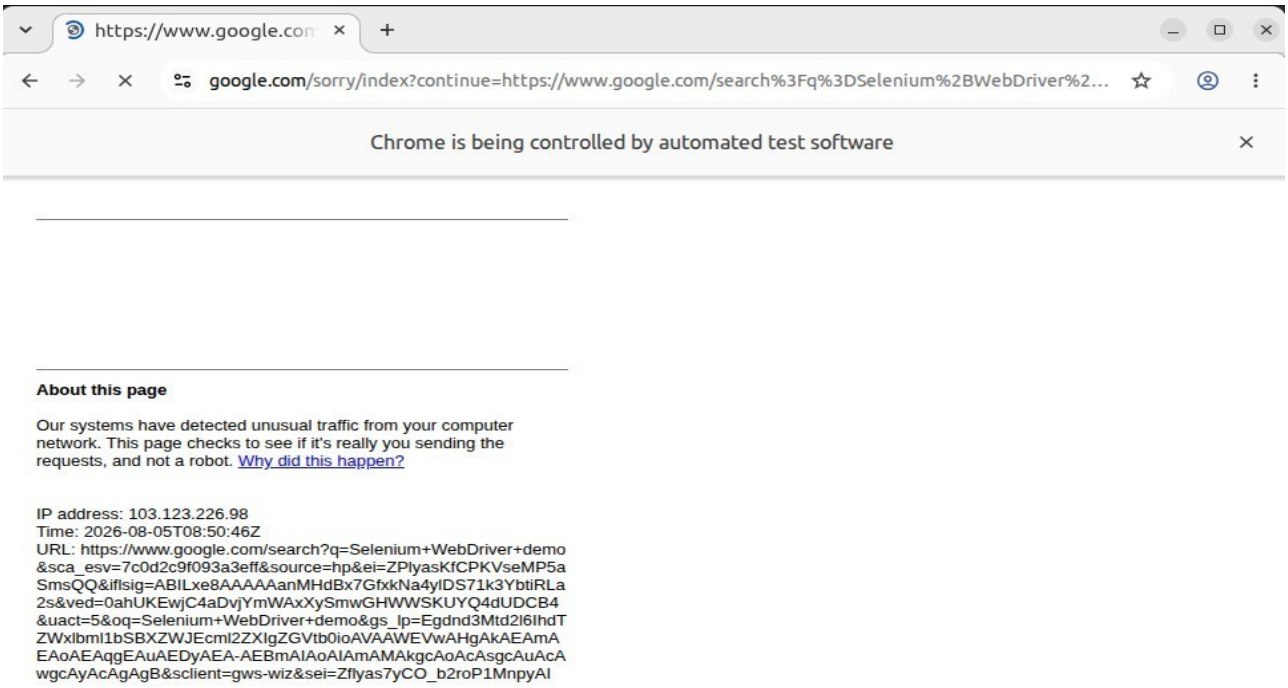
  pip install selenium
- **Download WebDriver**: Download the appropriate WebDriver for the browser you'll be using (e.g., ChromeDriver for Chrome). Ensure the WebDriver is in the system path.

1. Basic Demo :Automating a Google Search

```
apsit@apsit-HP-280-G3-MT:~$ python3 --version
Python 3.10.12
apsit@apsit-HP-280-G3-MT:~$ python3 --version
Python 3.10.12
apsit@apsit-HP-280-G3-MT:~$ pip3 install selenium
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: selenium in ./.local/lib/python3.10/site-packages (4.46.0)
Requirement already satisfied: trio<1.0,>=0.31.0 in ./.local/lib/python3.10/site-packages (from selenium) (0.33.0)
Requirement already satisfied: typing_extensions<5.0,>=4.15.0 in ./.local/lib/python3.10/site-packages (from selenium) (4.16.0)
Requirement already satisfied: urllib3[socks]<3.0,>=2.6.3 in ./.local/lib/python3.10/site-packages (from selenium) (2.7.0)
Requirement already satisfied: certifi>=2026.2.25 in ./.local/lib/python3.10/site-packages (from selenium) (2026.7.22)
Requirement already satisfied: websocket-client<2.0,>=1.8.0 in ./.local/lib/python3.10/site-packages (from selenium) (1.9.0)
Requirement already satisfied: trio-websocket<1.0,>=0.12.2 in ./.local/lib/python3.10/site-packages (from selenium) (0.12.2)
Requirement already satisfied: outcome in ./.local/lib/python3.10/site-packages (from trio<1.0,>=0.31.0->selenium) (1.3.0.post0)
Requirement already satisfied: exceptiongroup in ./.local/lib/python3.10/site-packages (from trio<1.0,>=0.31.0->selenium) (1.3.1)
Requirement already satisfied: sortedcontainers in ./.local/lib/python3.10/site-packages (from trio<1.0,>=0.31.0->selenium) (2.4.0)
Requirement already satisfied: idna in /usr/lib/python3/dist-packages (from trio<1.0,>=0.31.0->selenium) (3.3)
Requirement already satisfied: sniffio>=1.3.0 in ./.local/lib/python3.10/site-packages (from trio<1.0,>=0.31.0->selenium) (1.3.1)
Requirement already satisfied: attrs>=23.2.0 in ./.local/lib/python3.10/site-packages (from trio<1.0,>=0.31.0->selenium) (26.1.0)
Requirement already satisfied: wsproto>=0.14 in ./.local/lib/python3.10/site-packages (from trio-websocket<1.0,>=0.12.2->selenium) (1.3.2)
Requirement already satisfied: pysocks!=1.5.7,<2.0,>=1.5.6 in ./.local/lib/python3.10/site-packages (from urllib3[socks]<3.0,>=2.6.3->selenium
) (1.7.1)
Requirement already satisfied: h11<1,>=0.16.0 in ./.local/lib/python3.10/site-packages (from wsproto>=0.14->trio-websocket<1.0,>=0.12.2->selen
ium) (0.16.0)
apsit@apsit-HP-280-G3-MT:~$ pip3 show selenium
Name: selenium
Version: 4.46.0
Summary: Official Python bindings for Selenium WebDriver
Home-page: https://www.selenium.dev
Author:
Author-email:
License: Apache-2.0
Location: /home/apsit/.local/lib/python3.10/site-packages
                 ifi, trio, trio-websocket, typing_extensions, urllib3, websocket-client
Show Applications
apsit@apsit-HP-280-G3-MT:~$ touch google_search.py
```

google_search.py

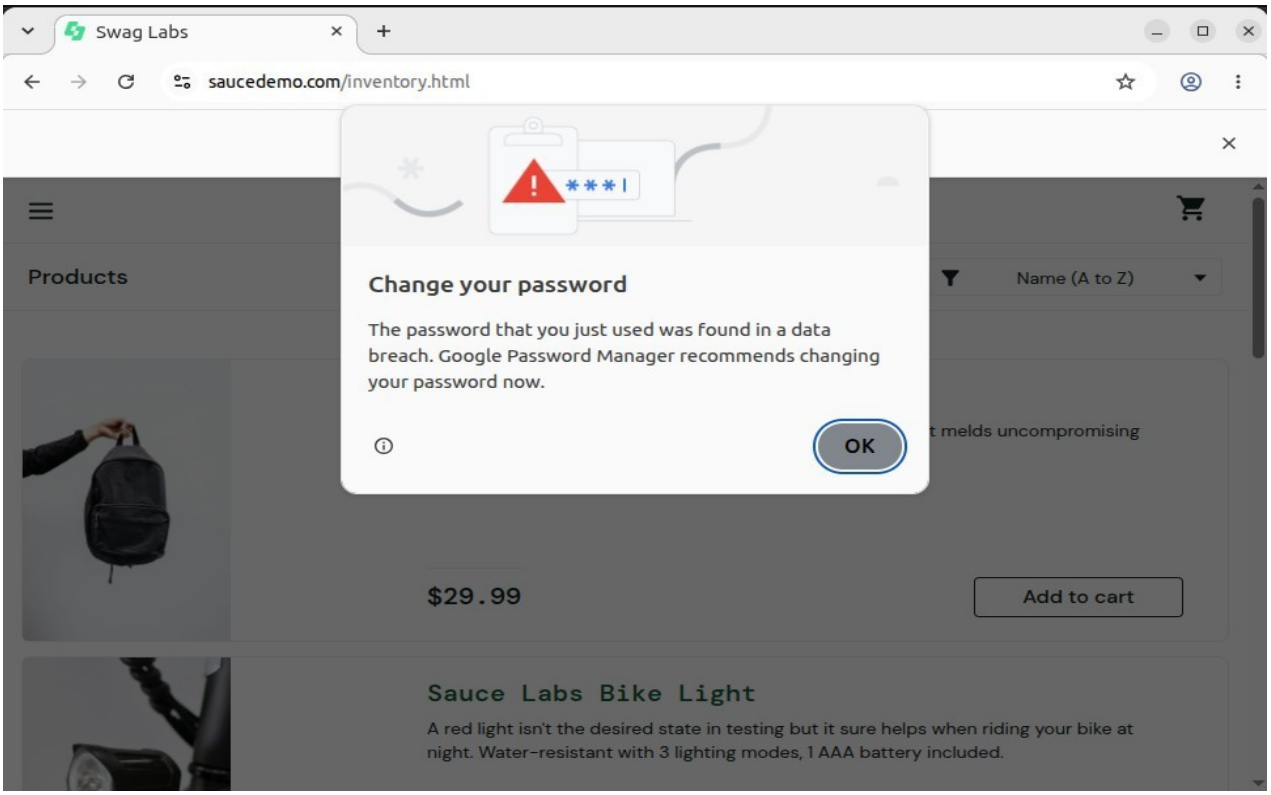
```python
# Import Selenium WebDriver
from selenium import webdriver
from selenium.webdriver.common.keys import Keys

# Set up WebDriver (Chrome in this case)
driver = webdriver.Chrome()

# Open Google
driver.get("https://www.google.com")

# Print page title
print(f"Page title: {driver.title}")

# Find the search box element using its name attribute value
search_box = driver.find_element("name", "q")

# Type a query into the search box
search_box.send_keys("Selenium WebDriver demo")

# Simulate pressing Enter
search_box.send_keys(Keys.RETURN)

# Wait for the results page to load and display the results
driver.implicitly_wait(25000)

# Print the title of the new page
print(f"New Page title: {driver.title}")

# Close the browser
driver.quit()
```



Demo 2: Automating a Form on a Website (Example: A login form)

```
apsit@apsit-HP-280-G3-MT: ~          apsit@apsit-HP-280-G3-MT: ~          ap
apsit@apsit-HP-280-G3-MT:~$ gedit login_demo.py
apsit@apsit-HP-280-G3-MT:~$ python3 login_demo.py
  File "/home/apsit/login_demo.py", line 9
    driver.get(&amp;quot;https://www.saucedemo.com/&amp;quot;)
               ^
SyntaxError: invalid syntax
apsit@apsit-HP-280-G3-MT:~$ python3 login_demo.py
Login successful
Page Title: Products
apsit@apsit-HP-280-G3-MT:~$ python3 login_demo.py
Login successful
Page Title: Products
apsit@apsit-HP-280-G3-MT:~$ python3 login_demo.py
Login successful
Page Title: Products
apsit@apsit-HP-280-G3-MT:~$
```

```python
# Import Selenium WebDriver
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

# Step 1: Set up WebDriver for Chrome
driver = webdriver.Chrome()

# Step 2: Open the demo login page
driver.get("https://www.saucedemo.com/")

# Step 3: Find the username and password input fields
username = driver.find_element(By.NAME, "user-name")
password = driver.find_element(By.NAME, "password")

# Step 4: Enter login credentials
username.send_keys("standard_user")
password.send_keys("secret_sauce")

# Step 5: Locate the login button and click it
submit_button = driver.find_element(By.NAME, "login-button")
submit_button.click()

# Step 6: Wait for the next page to load
time.sleep(3)

# Step 7: Verify login
try:
    products_title = driver.find_element(By.CLASS_NAME, "title")
    print("Login successful")
    print("Page Title:", products_title.text)
except Exception:
    print("Login failed")

# Step 8: Close the browser
```



Conclusion:

In conclusion, while Katalon and Cucumber offer specialized low-code and behavioral testing advantages, Selenium remains the foundational standard for robust, scalable web quality assurance through structured frameworks, dynamic waits, and continuous integration.